

Operating System Case Study: ZephyrOS

Steven Counterman, Sasha Crawford, and Patrick Smith

Department of Science, DeSales University

CS-314: Operating Systems

Prof. Evan Shimkanon

November 24, 2025

Operating System Case Study: ZephyrOS

ZephyrOS is a real-time operating system utilized for embedded devices or IoT gateways. While systems with necessary resources can rely on general-purpose operating systems such as Linux or Windows, smaller IoT devices face strict limitations in memory, power consumption, and processing capability. ZephyrOS is made to be flexible as an RTOS. A RTOS is an operating system that is specifically designed for systems that take in a large number of events. Furthermore, it prioritizes individual tasks as opposed to general operating systems that focus on multitasking or user interaction. (Upadhyay). ZephyrOS was created at a time when requirements for technology were vast, and there was a need for a variety of use cases. ZephyrOS is open source – this means that it makes it more flexible to adaptable to the changing times (*About the Zephyr Project*).

Motivations

The initial stages of ZephyrOS officially launched in 2016 and already received about 7,000 commits. “The project was started in 2014 and was publicly launched as a Linux Foundation project in 2016 with founding members Intel, Wind River, Synopsis, and NXP” (Nashif). The primary purpose of ZephyrOS was to enable large-scale collaboration and support evolving resource constraints and real-time capabilities. This operating system contributes greatly to shape and influence more RTOS to come; especially those for embedded systems and IoT devices. Zephyr was developed with a certain modular architecture that allows for flexibility and decisive choosing to only utilize what is necessary from existing Zephyr components. It is also extremely scalable across a wide variety of devices and hardware requirements (Nashif).

ZephyrOS is extremely successful as an RTOS of our time. In such a fast moving and evolving time we live in, ZephyrOS continues to be useful and relevant. The use cases vary so

greatly – from sensors or wearables to even wind turbines. The scalability of the OS as well as the flexibility makes Zephyr usable across the board.

Technology

ZephyrOS is primarily written in the C programming language. C is commonly used in embedded systems development due to how efficient it is, its low-level memory control, and its performance, all of which are critical in resource-constrained environments typical of RTOS. This allows Zephyr to run efficiently on microcontrollers, as well as any other devices that have limited hardware resources. Additionally, Zephyr natively supports applications written in C++, as well as Rust. This allows developers to use Rust's memory safety features alongside the efficiency of C and C++.

Memory

Memory management in Zephyr is designed to be extremely flexible and configurable to meet the needs of embedded devices. It supports dynamic memory allocation, memory pools, and stack-based memory, enabling developers to optimize memory usage depending on the hardware constraints (*Zephyr Project Overview*). Tools are included to detect memory leaks and manage fragmentation, which helps maintain reliability and predictable performance, which are both essential in RTOS.

Zephyr has a very modular approach to file systems, allowing for multiple file system structures to be used at the same time. Each instance is identified by a mount point, which acts as the disk volume name and is used by the file system library when formatting or mounting storage. This design allows developers to choose the most suitable file system for their application while supporting a variety of storage devices, making ZephyrOS highly versatile in embedded storage solutions.

Kernel

Zephyr's kernel is responsible for its core functions such as task scheduling, inter-process communication, and device management. It ensures real-time performance by prioritizing tasks and handling interrupts efficiently. One useful module for system administration is the shell subsystem, which provides a command line interface that allows developers and system admins to monitor system performance, control devices, and debug applications. Another interesting module is the Zephyr bus, also known as the zbus, which acts as a lightweight software bus for many-to-many message passing between threads and tasks. Zbus implements publish / subscribe patterns, allowing modules to communicate without being tightly coupled. This makes it a valuable tool for inter-thread communication, event-driven programming, and building small subsystems, such as a sensor producer sending data to multiple consumer threads. The module also addresses real-time concerns like latency and priority handling, including priority inversion and priority boosting through zbus (*Zephyr Project Documentation*).

Security

Security is a big focus for ZephyrOS, which is guided by a defined process developed by the Zephyr Security Subcommittee to ensure secure development, design, and certification. The OS emphasizes functional security through cryptographic functionality, code quality, and execution protection, including stack guards, memory, and thread separation. Zephyr leverages PSA Crypto with mbedTLS for standardized cryptographic operations and plans to support secure hardware modules such as TPMs and Trusted Execution Environments. Its monolithic design, where the kernel and applications are compiled into a single static binary, reduces the attack surface by eliminating dynamic loaders and function call context switches. Quality assurance is enforced through mandatory code reviews, static code analysis, and complexity

checks to maintain code correctness, readability, and security, while threat modeling identifies assets and potential vulnerabilities to inform countermeasures. The system also supports secure boot, over-the-air updates, device authentication, and secure storage; all configured through modular options to minimize unnecessary exposure. Zephyr's secure development process integrates automation, change management, and documentation to ensure traceability and consistency, while its security architecture defines safeguards at both system and module levels. Finally, the OS is designed to support security certification through well-documented evaluation targets, threat mitigation, and verification, allowing vendors to certify either the OS itself, a hardware platform running Zephyr, or a complete product including applications (*Zephyr Project Documentation*).

Performance

A large difference between ZephyrOS and other open-source RTOSs in its domain is that both FreeRTOS and ThreadX have been acquired by Amazon and Microsoft, respectively. This has created some speculation about whether they are truly going to be open source into the future, as both companies have a history of turning open-source projects into corporate takeovers. (IIoT World, 2023). Zephyr, being managed by the Linux Foundation, is focused on creating sustainability for open-source projects. The adaptive, modular use of Zephyr is causing significant preference for it over FreeRTOS and ThreadX, which are both harder to use and more robust in their implementation. Overall, Zephyr allows for more modular implementation compared to the simplicity and robustness of other embedded operating systems like FreeRTOS and ThreadX.

In terms of hardware, Zephyr is extremely modular. It supports RISC-V, x86, ARM, Xtensa, and the team is currently working on support for ARC processors. As an embedded

system, a peripheral is any hardware block that is not the CPU core itself. While peripherals on a typical operating system are usually I/O devices, Zephyr's peripherals include Battery Backed RAM, fuel gauges, cyclic redundancy units, along with many others. Key advantages of Zephyr in the context of hardware support are that it supports more than 450 community-supported boards from a variety of architectures and vendors, and it supports many built-in driver interfaces and libraries. This allows it to support sensors and devices of all kinds, including IP networking capability, Wi-Fi, and Bluetooth. (Berard, 2023).

Overall, Zephyr's reception has been significantly positive. As it remains open source, unlike others in the embedded realm, the modularity and accessibility of Zephyr prove to be its greatest features. In fact, Zephyr has been used on devices with as little as 8 KB of Flash memory and less than 5 KB of RAM, proving its modularity. Its robust and secure kernel supports multiple scheduling algorithms and has significant fault tolerance. Zephyr provides an emulation/simulation framework for testing before implementation, making it attractive to many people searching for a reliable operating system ecosystem. The widespread adoption of this operating system can be seen in the devices it runs on. Zephyr's documentation provides a list of these devices, such as pet trackers, livestock trackers, Google Chromebooks, circuit breakers, and even Vestas' wind turbines. A list of supported boards is also provided, including Raspberry Pi Pico, Intel UP Squared, ESP32, and more than 780 boards overall.

References

About the Zephyr Project. (n.d.). Zephyr. Retrieved November 21, 2025, from

<https://www.zephyrproject.org/learn-about/>.

Berard, Stephen. (2023, Mar. 28). *Why We Moved from FreeRTOS to Zephyr RTOS.* Zephyr.

<https://zephyrproject.org/why-we-moved-from-freertos-to-zephyr-rtos/>

Clarysse, Ivo. (2019, Nov. 22). *Zephyr – An Operating System for IoT.* Zephyr.

<https://www.zephyrproject.org/zephyr-an-operating-system-for-iot/>.

IIoT World. (2023, January 31). *FreeRTOS vs ThreadX vs Zephyr: The fight for true open source RTOS.* IIoT World.

Nashif, Anas (n.d.) *The Zephyr Story: How It Became a Self-Sustaining Ecosystem.*

Intel. Retrieved 2025 Nov. 23, from [https://www.intel.com/content/www/us/en/](https://www.intel.com/content/www/us/en/developer/articles/community/zephyr-story-how-became-self-sustaining-ecosystem.html)

[developer/articles/community/zephyr-story-how-became-self-sustaining-ecosystem.html](https://www.intel.com/content/www/us/en/developer/articles/community/zephyr-story-how-became-self-sustaining-ecosystem.html)

Zephyr Project Documentation. (n.d.) Zephyr. Retrieved November 23, 2025, from

<https://docs.zephyrproject.org/latest/index.html>

Zephyr Project Overview (n.d.) Zephyr. Retrieved November 23, 2025, from

[https://www.zephyrproject.org/wp-content/uploads/2025/06/Zephyr-Overview-](https://www.zephyrproject.org/wp-content/uploads/2025/06/Zephyr-Overview-20250626.pdf)

[20250626.pdf](https://www.zephyrproject.org/wp-content/uploads/2025/06/Zephyr-Overview-20250626.pdf)

Upadhyay, Rajkumar (2025, July 12). Geeks for Geeks.

<https://www.geeksforgeeks.org/operating-systems/real-time-operating-system-rtos>